PickLine

Pick, Convey, and Fill Cell - Project Documentation

Raspberry Pi 3B+ / macOS - Qt, Blender, MQTT, SQLite, OPC UA

October 2026

Contents

1. Overview 3

2. System Architecture 3

3. Platform and Networking 3

4. Component Reference 3

4.1 Soft PLC . . . 4

4.2 Plant Simulation . . . 4

4.3 Qt HMI . . . 4

4.4 Blender 3D Twin . . . 4

4.5 Live Web Dashboard . . . 4

4.6 SQLite Historian . . . 5

4.7 OPC UA Bridge . . . 5

5. MQTT Topic Contract 5

5.1 Sensors - plant to clients . . . 5

5.2 Actuators and commands . . . 5

6. Fill and Routing Logic 6

7. Fault Handling 6

8. Running the System 6

8.1 One-time setup . . . 6

8.2 Start the core . . . 6

8.3 Start the cell . . . 6

9. Full Testing Procedure 6

9.1 MQTT layer . . . 6

9.2 HMI . . . 7

9.3 Blender twin . . . 7

9.4 Dashboard and historian . . . 7

9.5 OPC UA . . . 7

9.6 Faults . . . 7

10. OPC UA Address Space 7

11. Extending to Real Hardware 8

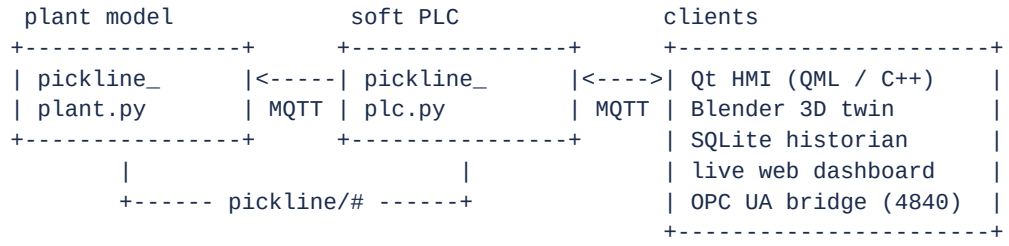
1. Overview

PickLine is a combined digital twin of a pick-and-place gantry, a conveyor belt, and two filling stations. The gantry picks an empty container from one of two bins and drops it onto the belt infeed. The belt carries the container to its end and routes it either straight to fill station A or left onto the side lane to fill station B. Each delivered container pours its fixed volume into the station tank; station A is full after 2 containers and station B after 5 (both adjustable). When a tank is full, the cell blocks until the container is taken out manually.

The project layers a realistic plant/PLC separation over an MQTT message bus and shows the same process through several clients: a Qt HMI, a Blender 3D twin, a live web dashboard, a SQLite historian, and a standard OPC UA interface.

2. System Architecture

The plant simulation and the soft PLC are separate processes. The PLC only ever sees tag values, never the simulation internals - the same boundary a real PLC has against a physical cell. Every component communicates exclusively through the pickline/# MQTT topic tree.



Component	File	Role
Soft PLC	plc/pickline_plc.py	State machine, routing and control decisions
Plant model	plant/pickline_plant.py	Gantry, conveyor, fill-station physics
Qt HMI	hmi/ (Main.qml, main.cpp, mqttbridge.*)	2D operator screen and controls
Blender twin	twin/ (PickLine_Twin.blend, blender_pickline.py)	3D animated twin and panel
Dashboard	dashboard/pickline_dashboard.py	Web production dashboard (port 8080)
Historian	historian/pickline_logger.py	SQLite telemetry logging
OPC UA bridge	opcua/opcua_server.py	Exposes tags as OPC UA nodes (port 4840)

3. Platform and Networking

- Raspberry Pi 3B+ (Debian trixie, ARM64) runs the plant, PLC and OPC UA bridge; macOS runs the Mosquitto MQTT broker and can run any client for development.
- The Pi and Mac are linked over a direct Ethernet link (no router). The Mac is 192.168.50.1 and the Pi is 192.168.50.2 on subnet 192.168.50.0/24.
- The broker listens on the Mac. Processes on the Pi set MQTT_BROKER=192.168.50.1; processes on the Mac use the default localhost.
- SSH access to the Pi is via ssh pi@192.168.50.2. Raspberry Pi Connect provides remote shell and screen sharing when a display is needed.

4. Component Reference

4.1 Soft PLC

The soft PLC runs a 10 Hz control loop and drives the plant through target topics. Its automatic sequence is:

```
TO_PICK -> LOWER_PICK -> GRIP -> LIFT_PICK -> TO_DROP -> LOWER_DROP
   -> RELEASE -> LIFT_DROP -> WAIT_EXIT -> WAIT_STATION -> TO_PICK ...
```

Routing is station-aware: when both stations are free it alternates straight/left one by one; when one station is full it routes only to the free station; when neither is free it waits in WAIT_STATION for a take-out. The pick side (down / up / alternate) and the route (auto / straight / left) are selectable, and a gap setting delays the next pick after a container enters a station.

4.2 Plant Simulation

The plant owns the whole cell and stays in lock-step. A 3-axis gantry (X 0-300, Y 0-220, Z 0-100 mm) travels at 140 mm/s with a two-finger gripper. The conveyor runs 0-400 mm along Y = 100 at an adjustable 20-250 mm/s (default 90). Fill stations sit at A (460, 100) and B (400, 240).

Parameter	Range	Default
Belt speed	20-250 mm/s	90 mm/s
Container volume	10-500 units	100 units
Fill A pickups (to full)	1-10	2
Fill B pickups (to full)	1-10	5
Gap between cycles	seconds	0.5 s
Grip open/close time	-	0.35 s

4.3 Qt HMI

A Qt 6 Quick application (QML + C++ with a Mosquitto-based MQTT bridge) draws the 2D operator screen: the belt, both fill tanks with live liquid levels, the bins, the moving container and gantry, and an industrial stack light. The side panel has Start / Stop / Reset, manual jog, volume and fill-speed sliders, a per-station Take out button, jam/clear controls, and a live state readout.

```
cd PickLine/hmi
cmake -S . -B build -DCMAKE_PREFIX_PATH=~/Qt/6.11.2/macos
cmake --build build
open build/appPickLineHMI.app
```

For automated screenshots the app reads PICKLINE_HMI_SHOT (path) and PICKLINE_HMI_SHOT_DELAY (seconds) at startup.

4.4 Blender 3D Twin

create_pickline_blend.py builds the 3D scene once; setup_pickline_text.py installs the RUN_PICKLINE.py loader, which executes blender_pickline.py at run time so edits to the live script are picked up without rebuilding the scene. The twin animates the gantry, jaws, conveyor, container, liquid columns and a four-lamp signal tower (green = running, yellow = paused, red = fault/estop, blue = both stations full), and adds a control panel for start/stop/reset/pause/estop, take-out, routing, gap and belt speed.

```
cd PickLine/twin
blender --background --python create_pickline_blend.py   # build scene once
blender --background --python setup_pickline_text.py     # install loader
```

4.5 Live Web Dashboard

A single-file Python HTTP server (port 8080) that subscribes directly to MQTT and keeps its own 60-sample history, so it works without the historian. It shows the machine state, both stations with live fill bars, total liquid dispensed, throughput (completed fills per minute), and a live history chart.

```
python3 dashboard/pickline_dashboard.py   # open http://localhost:8080
```

4.6 SQLite Historian

Logs the tag set to SQLite (pickline.db) every 0.5 s, including positions, belt, fill levels, counts, state, mode and totals. Run --stats for a quick summary.

```
python3 historian/pickline_logger.py
python3 historian/pickline_logger.py --stats
```

4.7 OPC UA Bridge

opcua/opcua_server.py mirrors the MQTT tag space into a standard OPC UA address space on opc.tcp://0.0.0.0:4840, so industrial clients (TwinCAT, CODESYS, UaExpert, SCADA) can monitor and control the cell. Read-only tags sync from MQTT; writable Settings and Commands publish back to MQTT. Two test clients are included: opcua_client_test.py (smoke test) and opcua_browse.py (full tree browse plus read/write).

```
pip install asyncua
python3 opcua/opcua_server.py                # opc.tcp://0.0.0.0:4840
python3 opcua/opcua_browse.py                # dump tree + values
python3 opcua/opcua_browse.py --read Status/state
python3 opcua/opcua_browse.py --write Settings/gap 2.5
python3 opcua/opcua_browse.py --write-bool Commands/take_a true
```

5. MQTT Topic Contract

5.1 Sensors - plant to clients

Topic	Meaning
pickline/x, y, z	Tool position, mm
pickline/grip, holding	Jaws and held-part state
pickline/part_x, part_y	Container location, mm
pickline/belt, speed, diverter	Conveyor state
pickline/placed, exited	Fed onto belt / entered a station
pickline/volume	Container capacity, units
pickline/fill/a/*	Station A level, state, count, filled, pickups, pouring
pickline/fill/b/*	Station B level, state, count, filled, pickups, pouring
pickline/fill/total	Total liquid dispensed, units

5.2 Actuators and commands

Topic	Meaning
pickline/target/x, y, z, grip, belt, diverter, speed	Gantry / conveyor targets
pickline/target/volume	Container capacity (10-500)
pickline/target/fill/a/pickups, fill/b/pickups	Containers to fill (1-10)
pickline/fill/a/take, fill/b/take	Take out a full container

Topic	Meaning
pickline/plant/reset	Clear belt, stations and counters
pickline/state, mode, side, decision, gap	PLC status
pickline/command/start, stop, reset, mode, side, gap	Control
pickline/fault/jam, fault/clear	Inject / clear a belt jam

Heartbeats: pickline/plant/alive and pickline/plc/alive publish once per second (retained). Watch everything with: mosquitto_sub -h localhost -t 'pickline/#' -v

6. Fill and Routing Logic

- Every container carries the same adjustable volume (default 100 units). A delivered container pours its whole volume into the station tank.
- Station A is full after 2 containers and station B after 5, so the two stations have different effective fill times while sharing one container volume.
- A station is only free when EMPTY; FILLING and FULL both block it. The cell waits in WAIT_STATION until the full container is taken out.
- When both stations are free the route alternates straight (A) / left (B) one by one; when one is full the other is used; when neither is free the cell waits.

7. Fault Handling

Fault	Cause	Effect / recovery
Belt jam	pickline/fault/jam or a stuck container	PLC enters FAULT with alarm BELT JAM; clear via fault/clear or Res
Overfill	volume lowered below a station current level	Station enters FAULT with alarm OVERFILL; clear via Take out

8. Running the System

8.1 One-time setup

```
brew services start mosquitto          # broker (macOS)
pip3 install --break-system-packages asyncua   # OPC UA bridge dependency (Pi)
```

8.2 Start the core

```
cd PickLine
python3 plant/pickline_plant.py         # machine simulation
python3 plc/pickline_plc.py             # soft PLC
python3 historian/pickline_logger.py    # optional, SQLite
```

8.3 Start the cell

```
mosquitto_pub -t pickline/command/mode  -m auto
mosquitto_pub -t pickline/command/side  -m alternate
mosquitto_pub -t pickline/command/gap   -m 0.5
mosquitto_pub -t pickline/command/start -m 1
```

On the Pi, prefix every process with MQTT_BROKER=192.168.50.1 so it reaches the broker on the Mac.

9. Full Testing Procedure

9.1 MQTT layer

- Start the broker, plant and PLC, then confirm live traffic: mosquitto_sub -h localhost -t 'pickline/#' -v
- Confirm heartbeats and state: pickline/state = IDLE, pickline/mode = auto, pickline/plant/alive ticking once per second.
- Start the cell with the commands above; state should advance through TO_PICK ... WAIT_EXIT ... WAIT_STATION.
- Take out a full container with pickline/fill/a/take and confirm the station returns to EMPTY and the cycle resumes.

9.2 HMI

- Build and launch the Qt app; verify the canvas tracks the gantry, container, belt and both liquid levels.
- Verify Start / Stop / Reset, manual jog, per-station Take out, and the stack light colours (green run, yellow pause, red estop/fault, blue full).

9.3 Blender twin

- Open PickLine_Twin.blend and run RUN_PICKLINE.py; the 3D scene should mirror the MQTT state and the panel buttons should publish commands.

9.4 Dashboard and historian

- Run the dashboard and open http://localhost:8080; confirm fill bars, totals and throughput update in real time.
- Run the historian and confirm pickline.db grows; use --stats to read the summary.

9.5 OPC UA

- Start the bridge: python3 opcua/opcua_server.py (endpoint 4840).
- Browse and read: python3 opcua/opcua_browse.py lists the whole tree with live values; --read Status/state reads one node.
- Write a setting: python3 opcua/opcua_browse.py --write Settings/gap 2.5 publishes pickline/command/gap and the plant/PLC apply it.
- Trigger a command: python3 opcua/opcua_browse.py --write-bool Commands/take_a true publishes pickline/fill/a/take.
- From a Windows/Linux PC, UaExpert can browse the same nodes at opc.tcp://192.168.50.2:4840 with security policy None.

9.6 Faults

- Inject a jam: mosquitto_pub -t pickline/fault/jam -m 1; state becomes FAULT, alarm BELT JAM; clear with pickline/fault/clear -m 1.
- Lower volume below a station level to trigger OVERFILL, then clear it with the station Take out button.

10. OPC UA Address Space

Under Objects > PickLine, the bridge exposes these folders. Settings and Commands are writable; Command booleans are edge-triggered and reset to False after handling.

Folder	Nodes
Status	state, mode, side, route_mode, decision, alarm, paused, estop
Gantry	x, y, z, grip, holding
Conveyor	belt, speed, diverter, part_x, part_y, part_on_line

Folder	Nodes
Production	placed, exited, volume, fill_total
FillA	level, state, count, filled, pickups
FillB	level, state, count, filled, pickups
Settings (write)	speed, volume, gap, fill_a_pickups, fill_b_pickups, mode, route, side
Commands (write)	start, stop, reset, pause, resume, estop, clear_belt, take_a, take_b

11. Extending to Real Hardware

To run the control on real hardware later, replace plc/pickline_plc.py with a CODESYS runtime on the Pi and keep the same tag names. The HMI, Blender twin, historian and dashboard continue to work unchanged because they only talk MQTT. The OPC UA bridge is the standard integration point for SCADA, TwinCAT or CODESYS-based supervision.